Pandas

Pandas is a Python library.

Pandas is used to analyze data.

Basic

- Introduction , Getting started, Pandas Series, DataFrames, Read_CSV, Read JSON, Analyze Data.

```
! pip install pandas

import pandas as pd

### Now Pandas is imported and ready to use:

mydataSet = {
    "cars" : ["BMW","Volvo", "Ford"],
    'passings': [2,3,7]
}
myvar = pd.DataFrame(mydataSet)
print(myvar)
```

```
    cars  passings
0    BMW         2
1  Volvo         3
2   Ford         7
```

```
myvar["cars"]
```

```
0      BMW
1    Volvo
2     Ford
Name: cars, dtype: object
```

```
# What is a series?
## A Pandas Series is like a column in a table.
## It is a one-dimensional array holding data of any type>

a = [1,7,2]
myvar = pd .Series(a)
print(myvar)
```

```
0    1
1    7
2    2
dtype: int64
```

```
# Labels
## if nothing else is specified, the values are labeled with their
index number. First value has index 0, second values has index 1 etc.
## This label can be used to access a specified value.

print(myvar[0])
print(myvar[2])
```

```
1
2
```

```
# Create Labels
## With the - index -- argument, you can name your own labes.

a = [1, 4, 8]
myvar = pd.Series(a, index = ["x","y","z"])

print(myvar)
```

```
x    1
y    4
z    8
dtype: int64
```

```
## when you have created labels, you can access an item by refering to
the label.

print(myvar["y"])
```

```
4
```

```
# DataFrames
## Data sets in Pandas are usually multi-dimensional tables , called
DataFrames.
## Series is like a column, a DataFrame is the whole table.

data = {
    "calories" : [420, 380, 390],
    "duration" : [50,40,45]
}

myvar = pd.DataFrame(data)
print(myvar)
```

```
   calories  duration
0       420        50
1       380        40
2       390        45
```

Pandas DataFrames

what is a DataFrame?

A Pandas DataFrame is a 2 dimensional data structure, like a 2 dimensional array , or a table with rows and columns.

```
data = {
    "calories" : [320,430,390],
    "duration" : [50,40,45]
}

# load data into a DataFrame object:

df = pd.DataFrame(data)
print(df)
```

```
   calories  duration
0       320        50
1       430        40
2       390        45
```

Locate Row

As you can see from the result above , the DataFrame is like a table with rows and columns.

Pandas use the - loc - attribute to return one or more specified rows(s)

```
print(df.loc[0])
```

```
calories    320
duration     50
Name: 0, dtype: int64
```

```
print(df.loc[2])
```

```
calories    390
duration     45
Name: 2, dtype: int64
```

```
### use a list of indexes:

print(df.loc[[0, 1]])

### when using [] , the result is a Pandas DataFrame.
```

```
   calories  duration
0       320        50
1       430        40
```

```
print(df.loc[0:1])
```

```
   calories  duration
0       320        50
1       430        40
```

Load files into a DataFrame

if your data sets are stored in a file, Pandas can load them into a DataFrame.

```
### load a comma separate file(CSV file) into DataFrame:
df = pd.read_csv('data.csv')
```

Pandas Read CSV

Read csv files

A simple way to store big data sets is touse CSV files(comma separate files).

CSV files contains plain text and is a well know format that can be read by everyone including Pandas.

in our examples we will be using a csv file called "data.csv"

```
df = pd.read_csv("placement .csv")

print(df.to_string()) # use to_string to print entire DataFrame.
```

```
     cgpa  package
0    6.89     3.26
1    5.12     1.98
2    7.82     3.25
3    7.42     3.67
4    6.94     3.57
5    7.89     2.99
6    6.73     2.60
7    6.75     2.48
8    6.09     2.31
```

```
9    8.31     3.51
10   5.32     1.86
11   6.61     2.60
12   8.94     3.65
13   6.93     2.89
14   7.73     3.42
15   7.25     3.23
16   6.84     2.35
17   5.38     2.09
18   6.94     2.98
19   7.48     2.83
20   7.28     3.16
21   6.85     2.93
22   6.14     2.30
23   6.19     2.48
24   6.53     2.71
25   7.28     3.65
26   8.31     3.42
27   5.42     2.16
28   5.94     2.24
29   7.15     3.49
30   7.36     3.26
31   8.10     3.89
32   6.96     3.08
33   6.35     2.73
34   7.34     3.42
35   6.87     2.87
36   5.99     2.84
37   5.90     2.43
38   8.62     4.36
39   7.43     3.33
40   9.38     4.02
41   6.89     2.70
42   5.95     2.54
43   7.66     2.76
44   5.09     1.86
45   7.87     3.58
46   6.07     2.26
47   5.84     3.26
48   8.63     4.09
49   8.87     4.62
50   9.58     4.43
51   9.26     3.79
52   8.37     4.11
53   6.47     2.61
54   6.86     3.09
55   8.20     3.39
56   5.84     2.74
57   6.60     1.94
```

58	6.92	3.09
59	7.56	3.31
60	5.61	2.19
61	5.48	1.61
62	6.34	2.09
63	9.16	4.25
64	7.36	2.92
65	7.60	3.81
66	5.11	1.63
67	6.51	2.89
68	7.56	2.99
69	7.30	2.94
70	5.79	2.35
71	7.47	3.34
72	7.78	3.62
73	8.44	4.03
74	6.85	3.44
75	6.97	3.28
76	6.94	3.15
77	8.99	4.60
78	6.59	2.21
79	7.18	3.00
80	7.63	3.44
81	6.10	2.20
82	5.58	2.17
83	8.44	3.49
84	4.26	1.53
85	4.79	1.48
86	7.61	2.77
87	8.09	3.55
88	4.73	1.48
89	6.42	2.72
90	7.11	2.66
91	6.22	2.14
92	7.90	4.00
93	6.79	3.08
94	5.83	2.42
95	6.63	2.79
96	7.11	2.61
97	5.98	2.84
98	7.69	3.83
99	6.61	3.24
100	7.95	4.14
101	6.71	3.52
102	5.13	1.37
103	7.05	3.00
104	7.62	3.74
105	6.66	2.82
106	6.13	2.19

```
107  6.33     2.59
108  7.76     3.54
109  7.77     4.06
110  8.18     3.76
111  5.42     2.25
112  8.58     4.10
113  6.94     2.37
114  5.84     1.87
115  8.35     4.21
116  9.04     3.33
117  7.12     2.99
118  7.40     2.88
119  7.39     2.65
120  5.23     1.73
121  6.50     3.02
122  5.12     2.01
123  5.10     2.30
124  6.06     2.31
125  7.33     3.16
126  5.91     2.60
127  6.78     3.11
128  7.93     3.34
129  7.29     3.12
130  6.68     2.49
131  6.37     2.01
132  5.84     2.48
133  6.05     2.58
134  7.20     2.83
135  6.10     2.60
136  5.64     2.10
137  7.14     3.13
138  7.91     3.89
139  7.19     2.40
140  7.91     3.15
141  6.76     3.18
142  6.93     3.04
143  4.85     1.54
144  6.17     2.42
145  5.84     2.18
146  6.07     2.46
147  5.66     2.21
148  7.57     3.40
149  8.28     3.67
150  6.30     2.73
151  6.12     2.76
152  7.37     3.08
153  7.94     3.99
154  7.08     2.85
155  6.98     3.09
```

```
156  7.38     3.13
157  6.47     2.70
158  5.95     3.04
159  8.71     4.08
160  7.13     2.93
161  7.30     3.33
162  5.53     2.55
163  8.93     3.91
164  9.06     3.82
165  8.21     4.08
166  8.60     3.98
167  8.13     3.60
168  8.65     3.52
169  9.31     4.37
170  6.22     2.87
171  8.01     3.76
172  6.93     2.51
173  6.75     2.56
174  7.32     2.99
175  7.04     3.50
176  6.29     3.23
177  7.09     3.64
178  8.15     3.63
179  7.14     3.03
180  6.19     2.72
181  8.22     3.89
182  5.88     2.08
183  7.28     2.72
184  7.88     3.14
185  6.31     3.18
186  7.84     3.47
187  6.26     2.44
188  7.35     3.08
189  8.11     4.06
190  6.19     2.69
191  7.28     3.48
192  8.25     3.75
193  4.57     1.94
194  7.89     3.67
195  6.93     2.46
196  5.89     2.57
197  7.21     3.24
198  7.63     3.96
199  6.22     2.33
```

```
# If you have a large DataFrame with many rows , Pandas will only
return the first 5 rows and the last 5 rows:

df = pd.read_csv("placement .csv")
print(df)
```

```
     cgpa  package
0    6.89     3.26
1    5.12     1.98
2    7.82     3.25
3    7.42     3.67
4    6.94     3.57
..    ...      ...
195  6.93     2.46
196  5.89     2.57
197  7.21     3.24
198  7.63     3.96
199  6.22     2.33

[200 rows x 2 columns]
```

max_rows

The number of rows returned is defined in Pandas option setting.

You can check your system's maximum rows with the pd.options.display.max_row statement

```
print(pd.options.display.max_rows)
```

```
60
```

```
### In my system the number is 60, which means that if the DataFrame
contains more then 60 rows , the print(df) statement will
### return only the headers and the first and last 5 rows.
### You can change the maximum rows number with the statement.

pd.options.display.max_rows = 9999

df = pd.read_csv("placement .csv")

print(df)
```

```
     cgpa  package
0    6.89     3.26
1    5.12     1.98
2    7.82     3.25
3    7.42     3.67
4    6.94     3.57
5    7.89     2.99
6    6.73     2.60
7    6.75     2.48
8    6.09     2.31
9    8.31     3.51
```

```
10   5.32     1.86
11   6.61     2.60
12   8.94     3.65
13   6.93     2.89
14   7.73     3.42
15   7.25     3.23
16   6.84     2.35
17   5.38     2.09
18   6.94     2.98
19   7.48     2.83
20   7.28     3.16
21   6.85     2.93
22   6.14     2.30
23   6.19     2.48
24   6.53     2.71
25   7.28     3.65
26   8.31     3.42
27   5.42     2.16
28   5.94     2.24
29   7.15     3.49
30   7.36     3.26
31   8.10     3.89
32   6.96     3.08
33   6.35     2.73
34   7.34     3.42
35   6.87     2.87
36   5.99     2.84
37   5.90     2.43
38   8.62     4.36
39   7.43     3.33
40   9.38     4.02
41   6.89     2.70
42   5.95     2.54
43   7.66     2.76
44   5.09     1.86
45   7.87     3.58
46   6.07     2.26
47   5.84     3.26
48   8.63     4.09
49   8.87     4.62
50   9.58     4.43
51   9.26     3.79
52   8.37     4.11
53   6.47     2.61
54   6.86     3.09
55   8.20     3.39
56   5.84     2.74
57   6.60     1.94
58   6.92     3.09
```

59	7.56	3.31
60	5.61	2.19
61	5.48	1.61
62	6.34	2.09
63	9.16	4.25
64	7.36	2.92
65	7.60	3.81
66	5.11	1.63
67	6.51	2.89
68	7.56	2.99
69	7.30	2.94
70	5.79	2.35
71	7.47	3.34
72	7.78	3.62
73	8.44	4.03
74	6.85	3.44
75	6.97	3.28
76	6.94	3.15
77	8.99	4.60
78	6.59	2.21
79	7.18	3.00
80	7.63	3.44
81	6.10	2.20
82	5.58	2.17
83	8.44	3.49
84	4.26	1.53
85	4.79	1.48
86	7.61	2.77
87	8.09	3.55
88	4.73	1.48
89	6.42	2.72
90	7.11	2.66
91	6.22	2.14
92	7.90	4.00
93	6.79	3.08
94	5.83	2.42
95	6.63	2.79
96	7.11	2.61
97	5.98	2.84
98	7.69	3.83
99	6.61	3.24
100	7.95	4.14
101	6.71	3.52
102	5.13	1.37
103	7.05	3.00
104	7.62	3.74
105	6.66	2.82
106	6.13	2.19
107	6.33	2.59

```
108  7.76     3.54
109  7.77     4.06
110  8.18     3.76
111  5.42     2.25
112  8.58     4.10
113  6.94     2.37
114  5.84     1.87
115  8.35     4.21
116  9.04     3.33
117  7.12     2.99
118  7.40     2.88
119  7.39     2.65
120  5.23     1.73
121  6.50     3.02
122  5.12     2.01
123  5.10     2.30
124  6.06     2.31
125  7.33     3.16
126  5.91     2.60
127  6.78     3.11
128  7.93     3.34
129  7.29     3.12
130  6.68     2.49
131  6.37     2.01
132  5.84     2.48
133  6.05     2.58
134  7.20     2.83
135  6.10     2.60
136  5.64     2.10
137  7.14     3.13
138  7.91     3.89
139  7.19     2.40
140  7.91     3.15
141  6.76     3.18
142  6.93     3.04
143  4.85     1.54
144  6.17     2.42
145  5.84     2.18
146  6.07     2.46
147  5.66     2.21
148  7.57     3.40
149  8.28     3.67
150  6.30     2.73
151  6.12     2.76
152  7.37     3.08
153  7.94     3.99
154  7.08     2.85
155  6.98     3.09
156  7.38     3.13
```

```
157  6.47     2.70
158  5.95     3.04
159  8.71     4.08
160  7.13     2.93
161  7.30     3.33
162  5.53     2.55
163  8.93     3.91
164  9.06     3.82
165  8.21     4.08
166  8.60     3.98
167  8.13     3.60
168  8.65     3.52
169  9.31     4.37
170  6.22     2.87
171  8.01     3.76
172  6.93     2.51
173  6.75     2.56
174  7.32     2.99
175  7.04     3.50
176  6.29     3.23
177  7.09     3.64
178  8.15     3.63
179  7.14     3.03
180  6.19     2.72
181  8.22     3.89
182  5.88     2.08
183  7.28     2.72
184  7.88     3.14
185  6.31     3.18
186  7.84     3.47
187  6.26     2.44
188  7.35     3.08
189  8.11     4.06
190  6.19     2.69
191  7.28     3.48
192  8.25     3.75
193  4.57     1.94
194  7.89     3.67
195  6.93     2.46
196  5.89     2.57
197  7.21     3.24
198  7.63     3.96
199  6.22     2.33
```

Pandas Read Json

Big data sets are often stored, or extracted as JSON

JSON is plain text, but has the format of an object, an is well known in the world of programming, including Pandas.

In our examples we will be using a JSON file called "data.json"

```
df = pd.read_json("data.json")

print(df.to_string())
```

```
     cgpa  package
0    6.89     3.26
1    5.12     1.98
2    7.82     3.25
3    7.42     3.67
4    6.94     3.57
5    7.89     2.99
6    6.73     2.60
7    6.75     2.48
8    6.09     2.31
9    8.31     3.51
10   5.32     1.86
11   6.61     2.60
12   8.94     3.65
13   6.93     2.89
14   7.73     3.42
15   7.25     3.23
16   6.84     2.35
17   5.38     2.09
18   6.94     2.98
19   7.48     2.83
20   7.28     3.16
21   6.85     2.93
22   6.14     2.30
23   6.19     2.48
24   6.53     2.71
25   7.28     3.65
26   8.31     3.42
27   5.42     2.16
28   5.94     2.24
29   7.15     3.49
```

30	7.36	3.26
31	8.10	3.89
32	6.96	3.08
33	6.35	2.73
34	7.34	3.42
35	6.87	2.87
36	5.99	2.84
37	5.90	2.43
38	8.62	4.36
39	7.43	3.33
40	9.38	4.02
41	6.89	2.70
42	5.95	2.54
43	7.66	2.76
44	5.09	1.86
45	7.87	3.58
46	6.07	2.26
47	5.84	3.26
48	8.63	4.09
49	8.87	4.62
50	9.58	4.43
51	9.26	3.79
52	8.37	4.11
53	6.47	2.61
54	6.86	3.09
55	8.20	3.39
56	5.84	2.74
57	6.60	1.94
58	6.92	3.09
59	7.56	3.31
60	5.61	2.19
61	5.48	1.61
62	6.34	2.09
63	9.16	4.25
64	7.36	2.92
65	7.60	3.81
66	5.11	1.63
67	6.51	2.89
68	7.56	2.99
69	7.30	2.94
70	5.79	2.35
71	7.47	3.34
72	7.78	3.62
73	8.44	4.03
74	6.85	3.44
75	6.97	3.28
76	6.94	3.15
77	8.99	4.60
78	6.59	2.21

79	7.18	3.00
80	7.63	3.44
81	6.10	2.20
82	5.58	2.17
83	8.44	3.49
84	4.26	1.53
85	4.79	1.48
86	7.61	2.77
87	8.09	3.55
88	4.73	1.48
89	6.42	2.72
90	7.11	2.66
91	6.22	2.14
92	7.90	4.00
93	6.79	3.08
94	5.83	2.42
95	6.63	2.79
96	7.11	2.61
97	5.98	2.84
98	7.69	3.83
99	6.61	3.24
100	7.95	4.14
101	6.71	3.52
102	5.13	1.37
103	7.05	3.00
104	7.62	3.74
105	6.66	2.82
106	6.13	2.19
107	6.33	2.59
108	7.76	3.54
109	7.77	4.06
110	8.18	3.76
111	5.42	2.25
112	8.58	4.10
113	6.94	2.37
114	5.84	1.87
115	8.35	4.21
116	9.04	3.33
117	7.12	2.99
118	7.40	2.88
119	7.39	2.65
120	5.23	1.73
121	6.50	3.02
122	5.12	2.01
123	5.10	2.30
124	6.06	2.31
125	7.33	3.16
126	5.91	2.60
127	6.78	3.11

```
128  7.93     3.34
129  7.29     3.12
130  6.68     2.49
131  6.37     2.01
132  5.84     2.48
133  6.05     2.58
134  7.20     2.83
135  6.10     2.60
136  5.64     2.10
137  7.14     3.13
138  7.91     3.89
139  7.19     2.40
140  7.91     3.15
141  6.76     3.18
142  6.93     3.04
143  4.85     1.54
144  6.17     2.42
145  5.84     2.18
146  6.07     2.46
147  5.66     2.21
148  7.57     3.40
149  8.28     3.67
150  6.30     2.73
151  6.12     2.76
152  7.37     3.08
153  7.94     3.99
154  7.08     2.85
155  6.98     3.09
156  7.38     3.13
157  6.47     2.70
158  5.95     3.04
159  8.71     4.08
160  7.13     2.93
161  7.30     3.33
162  5.53     2.55
163  8.93     3.91
164  9.06     3.82
165  8.21     4.08
166  8.60     3.98
167  8.13     3.60
168  8.65     3.52
169  9.31     4.37
170  6.22     2.87
171  8.01     3.76
172  6.93     2.51
173  6.75     2.56
174  7.32     2.99
175  7.04     3.50
176  6.29     3.23
```

```
177  7.09     3.64
178  8.15     3.63
179  7.14     3.03
180  6.19     2.72
181  8.22     3.89
182  5.88     2.08
183  7.28     2.72
184  7.88     3.14
185  6.31     3.18
186  7.84     3.47
187  6.26     2.44
188  7.35     3.08
189  8.11     4.06
190  6.19     2.69
191  7.28     3.48
192  8.25     3.75
193  4.57     1.94
194  7.89     3.67
195  6.93     2.46
196  5.89     2.57
197  7.21     3.24
198  7.63     3.96
199  6.22     2.33
```

Dictionary as JSON

JSON = Python Dictonary

if your JSON code is not in a file, but in a python Dictory. You can load it into a DataFrame directly:

```
data = {
  "Duration":{
    "0":60,
    "1":60,
    "2":60,
    "3":45,
    "4":45,
    "5":60
  },
  "Pulse":{
    "0":110,
    "1":117,
    "2":103,
    "3":109,
    "4":117,
```

```
    "5":102
  },
  "Maxpulse":{
    "0":130,
    "1":145,
    "2":135,
    "3":175,
    "4":148,
    "5":127
  },
  "Calories":{
    "0":409,
    "1":479,
    "2":340,
    "3":282,
    "4":406,
    "5":300
  }
}

df = pd.DataFrame(data)

print(df)
```

```
   Duration  Pulse  Maxpulse  Calories
0        60    110       130       409
1        60    117       145       479
2        60    103       135       340
3        45    109       175       282
4        45    117       148       406
5        60    102       127       300
```

Pndas - Analyzing DataFrames

Viewing the Data

One of the most used method for getting a quick overivew of the DataFrame, is the head() methond.

head() method returns the headers and a specified number of rows and strating from the top.

```
df = pd.read_csv('data.csv')
```

```
print(df.head())
print(df.head(10))

## There is also a tail() method for viewing the last rows of the
DataFrame.
## The tail() method  returns the headers and a specified of rows,
starting from the bottom.

df = pd.read_csv('data.csv')

print(dt.tail())

## Info About the Data
## The DataFrames objects has a method called  info(), that gives you
more  information about the data set.

#df = pd.read_csv('data.csv')

print(df.info)
```

```
<bound method DataFrame.info of     Duration  Pulse  Maxpulse  Calories
0         60    110       130       409
1         60    117       145       479
2         60    103       135       340
3         45    109       175       282
4         45    117       148       406
5         60    102       127       300>

#RangeIndex: 169 entries, 0 to 168
 # Data columns (total 4 columns):
```

Cleaning Data

Data Cleaning

Data Cleaning means fixing bad data in your data set

Bad data could be :

- Empty cells
- Data in wrong format
- Wrong data
- Duplicate

In this tutorial you will learn how to deal with all of them

Pandas - Cleanig Empty cells

Empty cells

Empty cells can potentially give you wrong result when you analyze data.

Remove Rows :

One wany to deal with empty cells is to remove rows that contain empty cells:

```
df = pd.read_csv('data.csv')

new_df = df.dropna()

print(new_df.to_string())

# By default , the dropna() method returns a new DataFrame, and will
not change the original



# If you want to change the original DataFrame , use the inplace =
True argument:

df = pd.read_csv('data.csv')
df.dropna(inplace = True)

print(df.to_string())

# Reaplace empty values

## Another way of dealing with empty cells is to insert a new value
instead.
## This way you do not have to delete entire rows just because of some
empty cells.
## The fillna() method allows us to replace empty cells with a value:
```

```
df = pd.read_csv('data.csv')

df.fillna(130, inplace = True)

# Replace only for specified Columns
## The example avove replaces all empty cells in the whole Data Frame.
## To only replace empty values for one column, specify the column
name for the DataFrame:
## Replace NULL ---- values in the " Calories" with the number 130:

df = pd.read_csv('data.csv')
df.fillna({"Calories" :130}, inplace = True)
```

Pandas - Cleaning Data of Wrong Format

Data of Wrong Format

Cells with data of wrong format can make it diffcult, or even impossible , to analyze data.

To fix it , You have two options: remove the rows , or convert all cells in the columns into the same format.

In our Data Frame , we have two cells with the wrong format , checkout row22 and row 26, the "date ' columns should be a string that represents date:

```
## let's try to convert all cells in the 'Date' column into dates:
## Pandas has a to_datetime() methond for this

df = pd.read_csv('data.csv')

df['Date'] = pd.to_datetime(df['Date'], format = 'mixed')


# As you can see from the result , the date in row 26 was fixed, but
the empty date in row 22 got a Nat(Not a time) value,
# in other words an empty value . One way to deal with empty values in
simple removing the entire row.

# Removing Rows
### The result from the converting in the example above gave us Nat
value , which can be handled as a Null value, and we cna remove the
### the row by using the  dropna() method.

## Example :
```

```
### Remove rows with a Null value in the 'Date' Column:

## subset ()

df.dropdna(subset = ['Date'], inplace = True)

# Pandas -Fixing Wrong Data
### Wrong data; Does not have to be 'empty cells' or 'wrong format',
it can just be wrong, like if someone registered '199' instead of
'1.99'
### Sometimes you can spot wrong data by looking at the data set,
because you have an expection of what is should be.
### if You take a look at our data set, You can see that in row 7, the
duration is 450, but for all the other rows the duration is between
30 and 60

# Repalcing values :
## One way to fix wrong values is to replace them with something else.
## In our example, it is most likely a typo, and the value should be
"45"  instead of "450" and we could just insert "45" in row 7:

df.loc[7, 'Duration'] = 45

### For samll data sets you might be able to replace the wrong data
one by one. but not for big data sets.
### To replace wrong data for larger data sets you can crated some
rules, e.g some boundaries for legal values, and replace any values
that are outside of the bundaries.

### Example :
### Loop through all values in the "Duration column"
### if the value is higher than 120, set it to 120:

for x in df.index:
    if df.loc[x, "Duration"]  > 120:
        df.loc[x, "Duration"] = 120


df = pd.read_csv('data.csv')

for x in df.index:
    if  df.loc[x, "Duration"] > 120:
        df.loc[x, "Duration"] = 120
print(df.to_string())

# Removing Rows
### Another way of handling data is to remove the rows that contais
wrong data.
### This way you do not have to find out what to replace them with,
and there is a good chance you do not need them to do your analyese.
```

```
for x in df.index:
    if df.loc[x, "Duration"] > 120:
        df.drop(x, inplace = True)

df = pd.read_csv('data.csv')

for x in df.index:
    if df.loc[x, "Duration"] > 120:
        dr.drop(x, inplace = True)

        ### Remember to include the inplce = True  - argument to make
the changes in the original DataFrame object instead  of returning a
compy
```

Pandas - Removing Duplicates :

Discovering Duplicates

Duplicate rows are rows that have been registered more than one time.

By taking a look at our test data set, we can assume that row 11 an 12 are duplicates.

To discover duplicates, We can use the duplicate() method.

The duplicated() method returns a Boolean values for each row:

```
# Returns True for every row that is a duplicate, otherwise False:

print(df.duplicated())

# Removing Duplicates:
## To remove duplicate , use the drop_duplicates() method.

df.drop_duplicates(inplace = True)

## Remember : The (inplace = True ) will make sure that the method
does not a new DataFrame, but it will remove add duplicates, but it
will remove all
## duplicates from the original DataFrame.

df = pd.read_csv('data.csv')

df.drop_dubplicates(inplace = True)

print(df.to_sting())
```

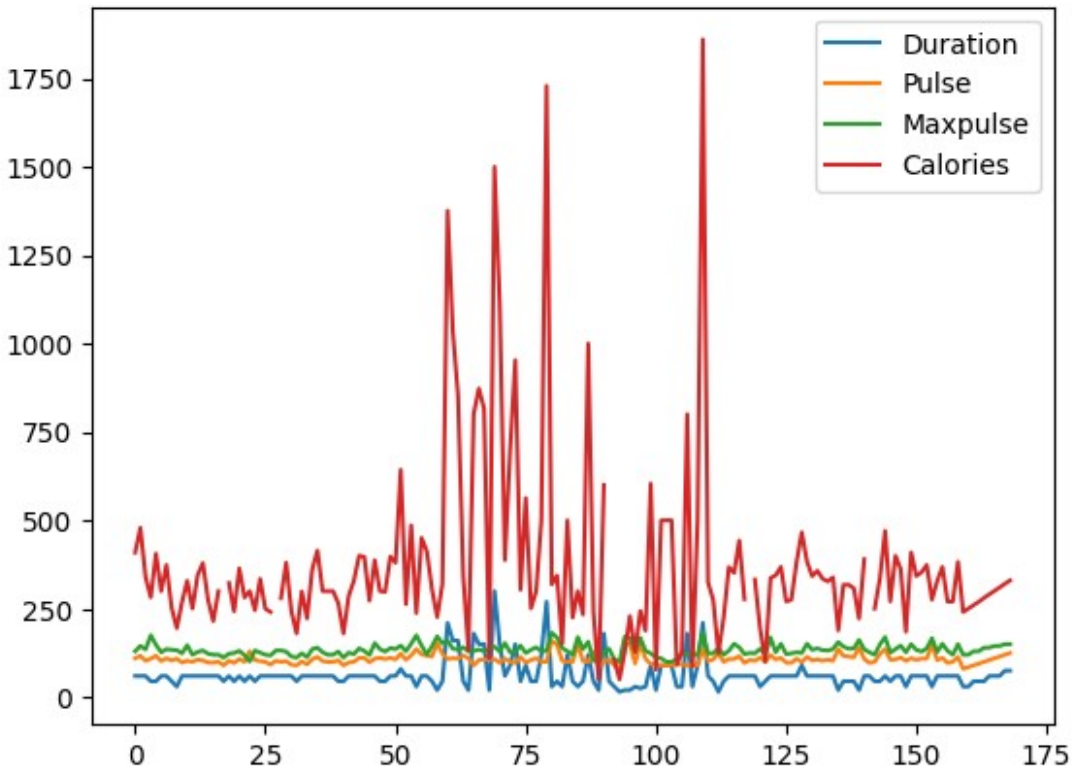
```
# Notice that row 12 has been removed from the result.

# Pandas - Data Correlations
## Finding Relationships
## A great aspect of the Pandas module is the corr() method.
## The corr() method calculates the relationship betweeen each column
in your data set.
## The examples in this page uses a csv file called " Data.csv"

df.corr()


## Reuslt Explained :

## The result of the corr() method is a table with a lot of numbes
that represents how well the relationship is between two columns.
## The number varies from -1 to 1.
## 1 mean that there is a 1 to 1 relationship (a perfect
correlation ), and for this data set, each time a value went up in the
first coulumn,
## the other one went up as well

## 0.9 also a good relationship , and if you incrase one value , the
other will probably incrase as well.
## -0.9  would be just as good relationship as 0.9 , but if you
incrase one value , the other will probably go down
## 0.2  means NOT a good relationship, meaning that if one value goes
up does not mean that the other will.

## What is a good correlation? It dependa on the use, but I think it
is safe to say you have to have at least 0.6 (or -0.6) to call it a
good correlation

"""
Perfect Correlation:
We can see that "Duration" and "Duration" got the number 1.000000,
which makes sense, each column always has a perfect relationship with
itself.

Good Correlation:
"Duration" and "Calories" got a 0.922721 correlation, which is a very
good correlation, and we can predict that the longer you work out, the
more calories you burn, and the other way around: if you burned a lot
of calories, you probably had a long work out.

Bad Correlation:
"Duration" and "Maxpulse" got a 0.009403 correlation, which is a very
bad correlation, meaning that we can not predict the max pulse by just
```

```
looking at the duration of the work out, and vice versa.
"""

# Pandas - Ploting

'''
Pandas use the plot() method to create diagrams.
We can use Pyplot, a submodule of the Matplotlib library to visulize
the diagram on the screen.


'''
df = pd.read_csv('.data.csv')
df.head()
```

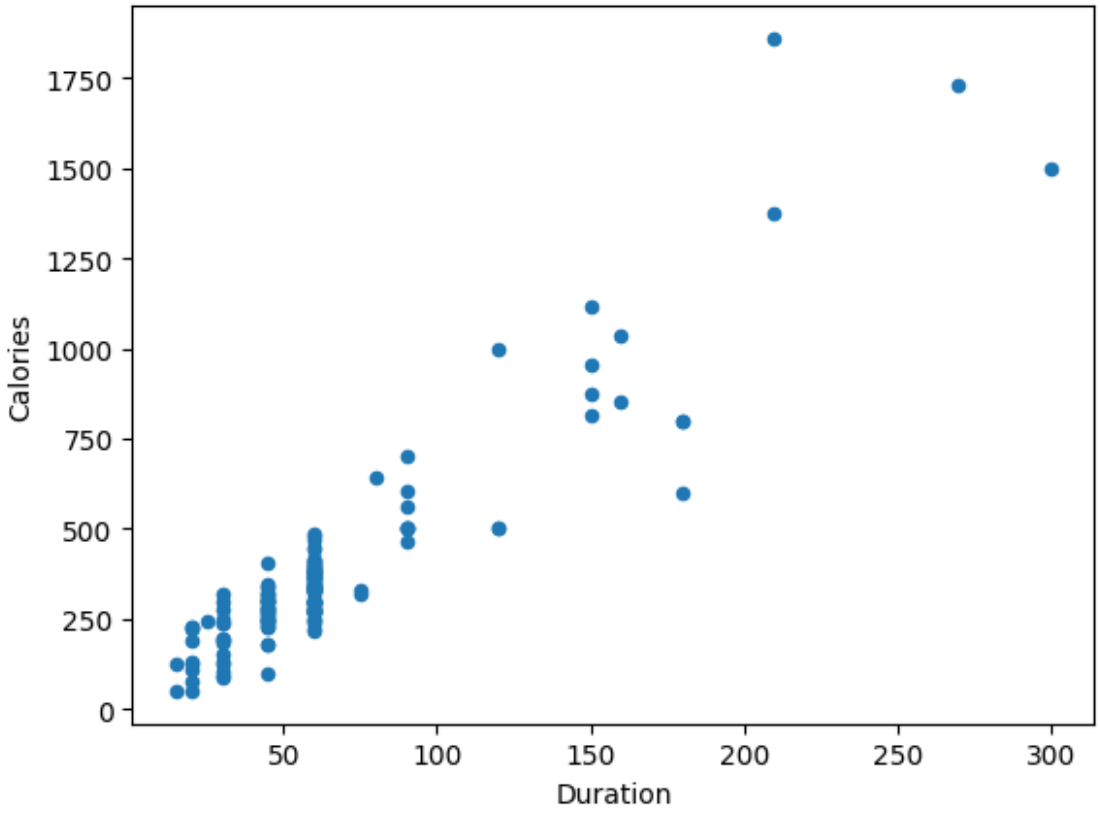
	Duration	Pulse	Maxpulse	Calories
0	60	110	130	409.1
1	60	117	145	479.0
2	60	103	135	340.0
3	45	109	175	282.4
4	45	117	148	406.0

```
df.plot()
```

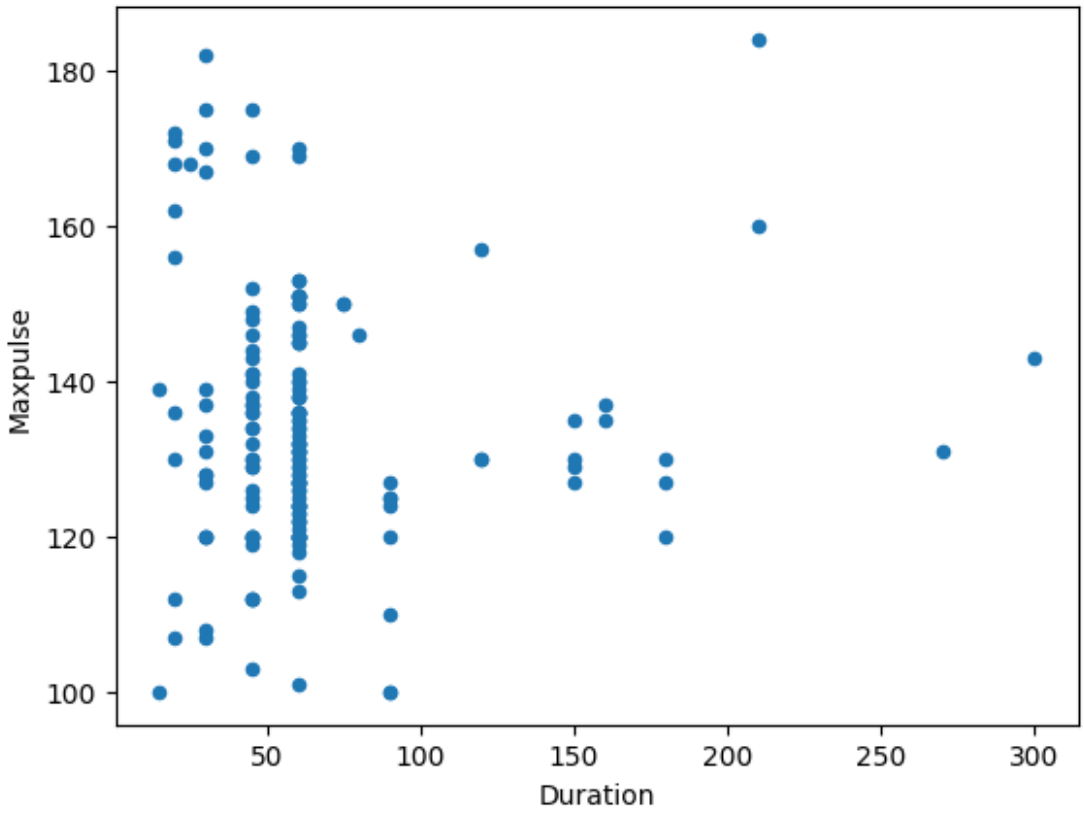
```
<Axes: >
```



```
# Scatter plot
"""
Specify that you want a scatter plot with the kind argument
 kind = 'scatter'

 A scatter plot needs an x and a y -axis
 In the example below we will use "Duration" for the x-axis and
"Calories" for the y-axis.
 Include the x and y argument like this :
 x = "Duration" and y = "Calories"

"""
import matplotlib.pyplot as plt
df.plot(kind= 'scatter', x = 'Duration', y = 'Calories')
plt.show()
```



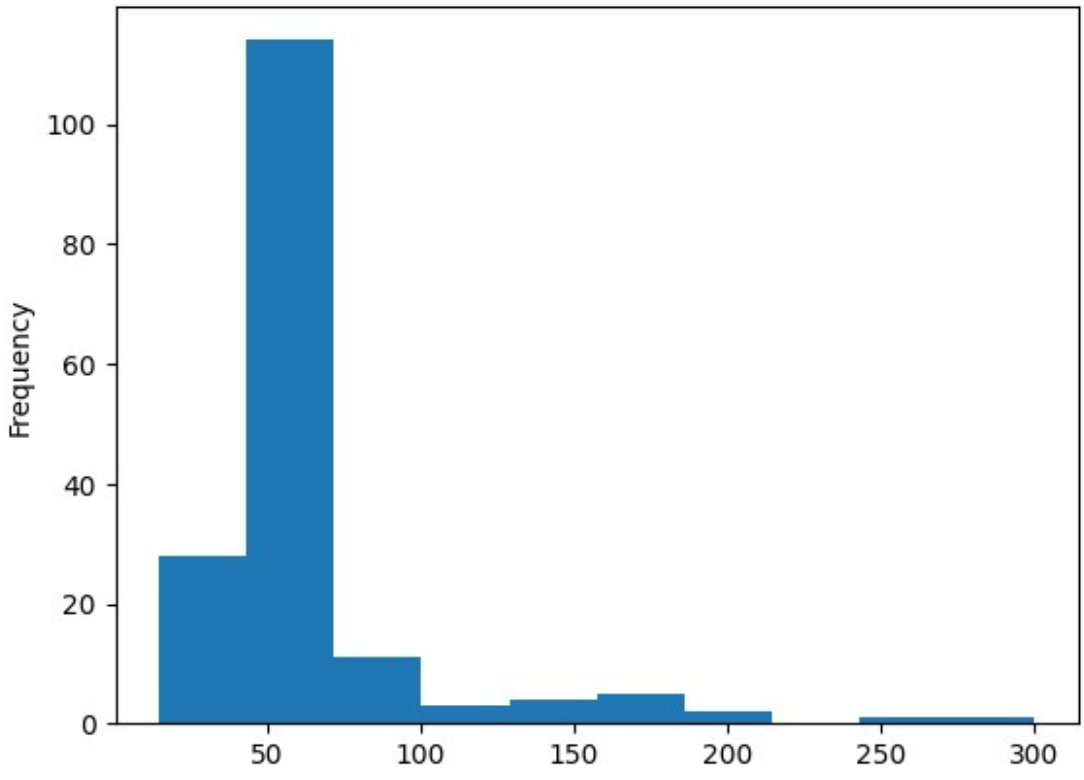

```
'''
A scatterplot where are no relationship between the columns:


'''
df =pd.read_csv(".data.csv")
df.plot(kind = 'scatter', x = "Duration", y = 'Maxpulse')
plt.show()
```



Cleaning Wrong Format:

```
# Histogram

'''
Use the kind argument to specify that you want a histogram:
kind = 'hist'
A histogram needs only one column.
A histogram shows us the frequencey of each interval. e.g many
workouts lasted between 50 and 60 minutes?

In the Example below we will use the "Duration" column to create the
histogram:

'''
df["Duration"].plot(kind = 'hist')
```

```
<Axes: ylabel='Frequency'>
```


Frequency
0
20
40
60
80
100
50
100
150
200
250
300